

BUSINESS REQUIREMENTS DOCUMENT

(Student Edition — Expanded & Annotated)

Event & Parking Reservation System

A Web-Based System for Event Seat and Parking Reservations

Suggested Tech Stack: ASP.NET Core Web API · SQL Server · HTML/CSS/JavaScript · Fetch API

Document Type: Business Requirements Document (BRD)

1. Project Overview

The Event & Parking Reservation System is a full-stack web application that lets customers browse upcoming events, reserve specific event seats, optionally reserve a parking slot at the same time, and keep track of all their bookings and notifications in one place. Administrators manage the events, parking layout, bookings and overall system statistics from a single dashboard.

As a student project, this brief mirrors a real client engagement: requirements are grouped into modules, each with its own data, rules and workflow, and you are expected to design the database, build the API, and connect it to a plain HTML/CSS/JavaScript frontend using the Fetch API — the same pattern used across all of your other full-stack coursework.

1.1 Why This Project Is a Good Fit for First-Year Students

- It builds on the same full-stack pattern (database → API → UI) used in other module projects, so prior skills transfer directly.
- The seat and parking selection grids give real practice with visual, state-driven UI — not just plain CRUD forms.
- Business rules around double-booking (seats and parking) give focused practice writing concurrency-safe validation logic.
- The scope divides cleanly across a team or a semester: seating, parking and booking can each be built and tested independently before being combined.

2. Objectives

By the end of this project, students should be able to demonstrate the following outcomes:

Objective	What This Demonstrates
Develop REST APIs using ASP.NET Core	Ability to design resource-based URLs and implement GET/POST/PUT/DELETE actions in ASP.NET Core Web API.
Build an interactive frontend using HTML, CSS and JavaScript	Ability to build a usable, visual UI (e.g. clickable seat/parking maps) using only vanilla JavaScript.
Integrate the frontend with Fetch API	Ability to consume a REST API asynchronously and update the DOM based on JSON responses.
Implement seat and parking reservation logic	Ability to design and enforce availability rules so no two customers can claim the same seat or slot.
Apply business rules and validations	Ability to translate written rules into server-side and client-side validation logic.

3. User Roles & Permissions

Two roles interact with the system. Keep the permission boundaries below in mind when designing your authentication/authorization logic.

3.1 Customer

- Browse events and view seat/parking availability, filterable by venue or category.
- Reserve one or more event seats.
- Optionally reserve a parking slot while booking.

- Pay for a booking (simulation) and view payment history.
- View own booking history and cancel a booking.
- View own notifications (confirmations, cancellations, reminders, updates).

3.2 Administrator

- Manage venues and event categories used across the system.
- Manage events (add, edit, delete) including venue, category, date, time and ticket price.
- Manage the seat map and parking slot layout for each event.
- View and manage all customer bookings and payments.
- View system-wide dashboard statistics across all modules.

4. Functional Modules

Each module below follows the same structure: a description, the features a student must implement, the business rules that govern it, a starter list of REST endpoints, and the database tables it relies on. Treat the endpoint tables as a minimum — you may add endpoints (e.g. pagination, filtering) as needed.

Module 1: Customer Management

Handles customer registration and self-service profile management. This module is usually built first because every booking depends on a valid Customer record.

Key Features

- Register a new customer (name, email, phone, password).
- Update own profile information.
- Search/filter customers by name or email (admin only).
- View a single customer's profile, including a summary of their bookings.

Business Rules

- Customer emails must be unique across all customers.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/customers/register	Register a new customer account.
GET	/api/customers/{id}	Get a single customer's profile.
PUT	/api/customers/{id}	Update a customer's own profile.
GET	/api/customers?search=	Search/filter customers (admin only).
DELETE	/api/customers/{id}	Deactivate a customer account (admin only).

Related Database Table(s)

- Customers

Module 2: Venue & Category Management

Gives administrators a place to maintain the master data that every event is built on: the venues events are held at, and the categories customers use to browse (e.g. Concert, Sports, Conference, Workshop).

Key Features

- Add, edit and delete venues (name, address, total capacity).
- Add, edit and delete event categories.
- List venues and categories for use in the Event Management dropdowns.
- Allow customers to filter the event listing by venue or category.

Business Rules

- A venue cannot be deleted while it has upcoming events scheduled at it (recommended safeguard).
- Only administrators can create, edit or delete venues and categories.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/venues	List all venues.
POST	/api/venues	Add a new venue (admin only).
PUT	/api/venues/{id}	Edit a venue (admin only).
DELETE	/api/venues/{id}	Delete a venue (admin only).
GET	/api/categories	List all event categories.
POST	/api/categories	Add a new category (admin only).

Related Database Table(s)

- Venues
- EventCategories

Module 3: Event Management

Lets administrators maintain the catalogue of events that customers can browse and book against, each linked to a venue and a category.

Key Features

- Add a new event (name, venue, category, date, time, ticket price).
- Edit an existing event's details.
- Delete an event (only if it has no active bookings — recommended safeguard).
- List and view events, including customer-facing search/filter by name, date, venue or category.

Business Rules

- An event must belong to exactly one venue and one category.
- An event's ticket price and seat map should not be changed after bookings exist, to avoid inconsistent pricing (recommended safeguard).

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/events	List all events, filterable by date, name, venue or category.
GET	/api/events/{id}	Get details for a single event.
POST	/api/events	Add a new event (admin only).
PUT	/api/events/{id}	Edit an event's details (admin only).

Method	Endpoint	Description
DELETE	/api/events/{id}	Delete an event (admin only).

Related Database Table(s)

- Events
- Venues
- EventCategories

Module 4: Seat Reservation

Displays a visual seat map for a chosen event. Customers click available seats to select them, see their running selection and total price, and submit them as part of a booking.

Key Features

- Render a visual seat map for the selected event (e.g. a grid of seat buttons, color-coded by status).
- Allow a customer to click and select one or more available seats.
- Show the selected seat numbers and running total ticket price as selections change.
- Prevent booked seats from being selected or clicked.

Business Rules

- A seat can only be booked once.
- Customers cannot book the same seat twice.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/events/{eventId}/seats	Get the full seat map with status (Available/Booked) for an event.
POST	/api/bookings/{bookingId}/seats	Attach selected seats to a booking (rejects already-booked seats).

Related Database Table(s)

- Seats
- BookingSeats

Module 5: Parking Reservation (Optional During Booking)

Displays a visual parking layout for the event and lets a customer optionally reserve a single slot as part of the same booking flow.

Key Features

- Render a visual parking layout for the selected event, showing available and occupied slots.
- Allow a customer to select at most one available parking slot while booking.
- Prevent occupied slots from being selected.
- Allow a booking to be completed with no parking slot selected (parking is optional).

Business Rules

- One parking slot can only be reserved by one customer for an event.
- Parking reservation is optional — a booking must not require a parking slot to complete.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/events/{eventId}/parking-slots	Get the parking layout with status for an event.
POST	/api/bookings/{bookingId}/parking	Reserve a parking slot for a booking (rejects if already reserved).
DELETE	/api/bookings/{bookingId}/parking	Remove a parking reservation from a booking before it's finalized.

Related Database Table(s)

- ParkingSlots
- ParkingReservations

Module 6: Booking Management

Ties seat selection and (optional) parking selection together into a single booking record, and lets customers track and cancel their own bookings.

Key Features

- Create a booking for an event, combining one or more selected seats and an optional parking slot.
- Generate a unique, human-readable booking number (e.g. BKG-2026-000123).
- View own booking history with status and details.
- Cancel a booking, freeing up its seats and parking slot for other customers.

Business Rules

- A booking must contain at least one seat.
- Cancelling a booking must release all of its seats and its parking slot (if any) back to Available status.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/bookings	Create a new booking (seats + optional parking).
GET	/api/bookings/customer/{customerId}	View a customer's own booking history.
GET	/api/bookings/{id}	Get full details of a single booking.
DELETE	/api/bookings/{id}	Cancel a booking and release its seats/parking (owner only).
GET	/api/bookings?eventId=	List all bookings for an event (admin only).

Related Database Table(s)

- Bookings
- BookingSeats
- ParkingReservations

Module 7: Payment Processing (Simulation)

Simulates payment for a booking — no real payment gateway is required. Customers pay for their seats (plus a parking fee, if reserved) and can view a payment history and download a receipt.

Key Features

- Calculate the total amount due from selected seats' ticket price plus a parking fee, if a slot was reserved.
- Mark a booking's payment as completed (simulation — no real transaction).
- View own payment history across all bookings.
- Generate/download a simple receipt for a completed payment.

Business Rules

- A booking cannot be marked Confirmed until its payment is completed.
- A payment cannot be recorded twice for the same booking.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/bookings/{id}/payment	Get the amount due and payment status for a booking.
POST	/api/bookings/{id}/payment	Mark a booking's payment as completed (simulation).
GET	/api/payments/customer/{customerId}	View a customer's payment history.
GET	/api/payments/{id}/receipt	Generate/download a receipt.

Related Database Table(s)

- Payments

Module 8: Notifications

Centralises system-generated messages so customers always know when something relevant to their bookings has happened — confirmation, cancellation, upcoming event reminders, and event updates. Notifications only need to be stored and displayed; real-time delivery (e.g. push/SignalR) is not required.

Key Features

- Automatically create a notification whenever a relevant booking event occurs (confirmed, cancelled, paid) or an event is updated.
- List a customer's notifications, newest first.
- Mark a notification as read.

Business Rules

- A customer should only ever see their own notifications.
- Notifications should be triggered from the relevant module's action (e.g. booking creation, payment completion), not created manually by the customer.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/notifications/customer/{customerId}	List a customer's notifications.
PUT	/api/notifications/{id}/read	Mark a notification as read.

Related Database Table(s)

- Notifications

5. Dashboards

5.1 Customer Dashboard

- Upcoming bookings.
- Reserved parking (if any).
- Recent payments.
- Unread notifications.

5.2 Administrator Dashboard

- Total events.
- Total bookings.
- Available seats (system-wide or per event).
- Occupied parking slots.
- Total revenue collected.
- Total customers.

 **Learning tip:** A dashboard is just several read-only GET endpoints (often one 'summary' endpoint that returns aggregated counts) rendered as cards on the frontend — you do not need a separate dashboard table in the database.

6. Consolidated Business Rules

These rules must be enforced primarily in the API layer (never trust client-side validation alone):

#	Rule	Where to Enforce It
1	A seat can only be booked once.	POST /api/bookings/{bookingId}/seats — check seat status before inserting, inside a transaction.
2	One parking slot can only be reserved by one customer for an event.	POST /api/bookings/{bookingId}/parking — check slot status before inserting, inside a transaction.
3	Customers cannot book the same seat twice.	Same check as Rule 1 — validate against existing BookingSeats records for that event.
4	Parking reservation is optional.	POST /api/bookings — allow booking creation to succeed with no parking slot supplied.
5	A booking must contain at least one seat.	POST /api/bookings — reject the request if the seat list is empty.
6	An event must belong to exactly one venue and one category.	POST/PUT /api/events — require VenueId and CategoryId and validate they exist.
7	A booking cannot be marked Confirmed until its payment is completed.	POST /api/bookings/{id}/payment — only flip booking status after payment succeeds.

7. REST API & Documentation Requirements

Build full CRUD APIs for Customers, Venues, Categories, Events, Seats, Parking Slots, Bookings, Payments and Notifications, following standard REST conventions:

- Use plural, resource-based URLs (e.g. /api/events, not /api/getEvents).
- Return proper HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict (for double-booking attempts).
- Use DTOs to control exactly what data is sent to the frontend (never expose password hashes or internal-only fields).
- Document every endpoint using Swagger/OpenAPI so you can test each one before wiring up the UI.
- Consider JWT-based authentication so the API knows whether a request is from a Customer or an Administrator.

8. Frontend Requirements

The frontend must be built using HTML, CSS and vanilla JavaScript only (no frameworks such as React, Angular or Vue), and must talk to the backend exclusively through the Fetch API.

- Provide search on event listings (by name, date, venue or category) and on the customer list (admin only).
- Build the seat map and parking layout as clickable grids that visually distinguish Available, Selected and Booked/Occupied states.
- Use responsive layout techniques (CSS Grid/Flexbox and media queries) so the portal works on both desktop and mobile.
- Validate forms on the client side for immediate feedback, in addition to server-side validation.
- Show a confirmation dialog before finalizing a booking, before submitting payment, and before cancelling a booking.
- Show clear loading and error states for every Fetch call, including a friendly message if a seat or slot was taken moments before submission.

9. Database Design

The tables below are a suggested starting schema. You are encouraged to refine column names/types and add audit fields (CreatedAt, UpdatedAt) as you design your own ER diagram.

Table	Purpose	Key Relationships
Customers	One row per customer account.	Referenced by Bookings, Payments and Notifications via CustomerId (FK).
Venues	One row per physical venue (name, address, capacity).	Referenced by Events.
EventCategories	Master list of event categories (Concert, Sports, etc.).	Referenced by Events.
Events	One row per event (name, venue, category, date, time, ticket price).	FK → Venues, FK → EventCategories; referenced by Seats, ParkingSlots and Bookings.
Seats	One row per physical seat belonging to an event's seat map.	FK → Events; referenced by BookingSeats once booked.

Table	Purpose	Key Relationships
ParkingSlots	One row per parking slot belonging to an event's parking layout.	FK → Events; referenced by ParkingReservations once reserved.
Bookings	One row per booking, with a generated booking number and status.	FK → Customers, FK → Events; referenced by Payments.
BookingSeats	Join table linking a booking to one or more seats.	FK → Bookings, FK → Seats.
ParkingReservations	One row per parking slot reserved as part of a booking.	FK → Bookings, FK → ParkingSlots.
Payments	One row per payment recorded against a booking.	FK → Bookings.
Notifications	One row per notification generated for a customer.	FK → Customers.

10. Non-Functional Requirements

These were not explicit in the original brief but are expected of any production-style student project:

Category	Expectation
Security	Passwords hashed (never stored in plain text); role-based authorization on admin-only endpoints.
Concurrency	Seat and parking reservation checks must be safe against two customers submitting at nearly the same time (use a transaction or a unique constraint).
Usability	Consistent navigation, clear feedback messages, and confirmation dialogs before irreversible actions.
Maintainability	Use a layered architecture (Controllers → Services → Repositories) rather than putting logic directly in controllers.
Portability	The SQL script and README must allow another developer to set up the project from scratch.

11. Suggested Build Order (For a Solo or Small-Team Project)

- Design the full database schema and relationships before writing any C# code.
- Build the Customer Management module end-to-end (DB → API → UI) first — every booking depends on it.
- Build Venue & Category Management next, since Event Management depends on both as reference data.
- Build Event Management, since seats and parking slots both belong to an event.
- Implement Seat Reservation, including the visual seat map and the double-booking check.
- Implement Parking Reservation using the same pattern as seats, then combine both into Booking Management.
- Add Payment Processing once bookings can be created, so there's a booking total to pay against.
- Add Notifications last, since it depends on booking/payment/event actions from other modules already existing.
- Build the dashboards last — they only aggregate data that already exists in the other tables.
- Finish with README, SQL script export and a final walkthrough test of every business rule, including a simulated double-click on the same seat.

12. Deliverables

Deliverable	Details
Source Code	Full ASP.NET Core Web API solution and frontend (HTML/CSS/JS), in a Git repository.
SQL Database Script	A script that creates the full schema and seeds minimal sample data (e.g. one event with seats and parking slots).
REST API	All CRUD endpoints described in Section 4, secured and returning correct status codes.
Frontend Application	Responsive HTML/CSS/JS pages for every module, including the seat and parking maps, wired to the API via Fetch.
Swagger Documentation	Auto-generated OpenAPI docs covering every endpoint, with example requests/responses.
README File	Setup instructions, tech stack, ER diagram, and a short description of each module.